

Cell 3 — EDA (Lab1 style) (Notebook cell index: 6)

This cell performs exploratory data analysis: HTML stripping, length features, class distribution, word-length distributions, a word cloud, and top token frequencies.

Granular line-by-line breakdown

Each block quotes the exact code, then names **sub-expressions** (variables, brackets, methods, arguments) and what they do, why they appear, and what breaks if you remove or misuse them.

Function `strip_html_noise`

```
def strip_html_noise(text):
```

- **def strip_html_noise(text):** — Defines a function named `strip_html_noise` with one parameter `text` (one headline cell). **Why:** You need a reusable rule for cleaning each row. **If missing:** You would duplicate the same cleaning logic inline.

```
    text = str(text)
```

- **text** (left side) — The local variable holding the string being cleaned.
- **str(...)** — Built-in constructor: converts `text` to a Python `str`. **Why:** CSV cells may be read as non-string types; `str` avoids errors and guarantees string methods below work. **If omitted:** Rare cells could raise `TypeError` on `html.unescape` or `re.sub`.

```
    text = html.unescape(text)
```

- **html** — Module imported earlier (`import html`).
- **html.unescape** — Function that replaces HTML entities (`&`, ` `, ...) with real characters. **Why:** Counts and tokens should reflect readable words, not encoded symbols. **If omitted:** Length and token counts treat entities as long noisy tokens.

```
    text = re.sub(r'<[^>]+>', ' ', text)
```

- **re** — Regular-expression module.
- **re.sub(pattern, replacement, string)** — Finds all matches of `pattern` in `string` and replaces them with `replacement`.
- **r'<[^>]+>'** — Raw string pattern: `<` then non-`>` characters then `>` → HTML/XML tags. **Why:** Tags are not topical words for EDA. **If omitted:** Words like `br` from `<br>` inflate vocabulary.

```
    text = re.sub(r'\s+', ' ', text).strip()
```

- **r'\s+'** — One or more whitespace characters (spaces, tabs, newlines).
- **Replacement ' '** — Collapses runs of whitespace to a single space.
- **.strip()** — Removes leading/trailing whitespace. **Why:** Consistent tokenization when splitting on space later. **If omitted:** Extra spaces create empty tokens or skew word counts.

```
    return text
```

- **return** — Sends the cleaned string back to the caller (`apply`). **If omitted:** Function returns `None`; every row becomes useless.
-

Copy and derived columns

```
eda_df = train_df.copy()
```

- **train_df** — Training DataFrame loaded earlier (column `text` = headline, `label` = topic).
- **.copy()** — DataFrame method: creates a separate table in memory. **Why:** Changes to `eda_df` do not mutate `train_df`, which modeling cells still need in raw form. **If omitted:** Risk of accidentally changing `train_df` when adding EDA columns.

```
eda_df['clean_eda'] = eda_df['text'].apply(strip_html_noise)
```

- **eda_df** — The exploratory copy of the training data.
- **eda_df['clean_eda']** — **Column selection / assignment target.** Creates a new column named `clean_eda` (or overwrites it). **Why:** Stores HTML-stripped text separately from original text.
- **=** — Assigns the result of the right-hand side into that column for every row.
- **eda_df['text']** — **Series** of raw headline strings (one per row).
- **.apply(strip_html_noise)** — **Series method:** calls `strip_html_noise` once per row, passing that row's text as text. **Why:** Vectorizes “clean every headline” without writing a Python for loop. **If you used apply wrong:** e.g. called `strip_html_noise(eda_df['text'])` — would pass the whole Series and usually error or behave incorrectly.
- **strip_html_noise** — Function **reference** (no parentheses): `apply` invokes it per row. **If you wrote strip_html_noise():** Would call the function once with no argument and pass its return value (wrong).

```
eda_df['char_len'] = eda_df['clean_eda'].str.len()
```

- **eda_df['clean_eda']** — Series of cleaned strings.
- **.str** — **String accessor:** exposes vectorized string ops on the whole column.
- **.len()** — Character length of each string. **Why:** Feature for distribution plots / sanity checks. **If omitted:** You lose headline-length insight.

```
eda_df['word_len'] = eda_df['clean_eda'].str.split().str.len()
```

- **.str.split()** — Splits each headline on whitespace → each cell becomes a **list of words**.
- **.str.len()** (second `.str`) — On lists-in-cells context in pandas, counts list length → word count per headline. **Why:** Word counts matter for choosing sequence length in RNNs and spotting outliers.

Preview table

```
display(eda_df[['clean_eda', 'label', 'char_len', 'word_len']].head(3))
```

- **display** — Jupyter/IPython function: renders rich output (nice HTML table).
- **eda_df[['...']]** — **Double brackets:** selects **multiple** columns → a smaller DataFrame.
- **'clean_eda', 'label', 'char_len', 'word_len'** — Which columns to show side by side.
- **.head(3)** — First 3 rows only. **Why:** Quick visual QA without printing thousands of rows.

Class distribution plot

```
plt.figure(figsize=(8, 4))
```

- **plt** — matplotlib.pyplot alias.
- **figure** — Creates a new figure.
- **figsize=(8, 4)** — Width and height in inches. **Why:** Readable bar chart size.

```
sns.countplot(data=eda_df, x='label', order=eda_df['label'].value_counts())
```

- `sns` — Seaborn.
- `countplot` — Bar chart of row counts per category.
- `data=eda_df` — Which table to read from.
- `x='label'` — Category axis = topic column.
- `order=...` — Controls bar order (here: most frequent class first).
 - `eda_df['label']` — Series of class labels.
 - `.value_counts()` — Counts per class, sorted descending.
 - `.index` — Class names in that sorted order. **Why:** Consistent, interpretable ordering.

```
plt.title('Class Distribution')
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

- `title` — Plot title text.
- `xticks(rotation=20)` — Rotates x-axis labels so long topic names do not overlap.
- `tight_layout()` — Adjusts padding so labels fit.
- `show()` — Displays the figure in the notebook.

Histogram and boxplot (two panels)

```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
```

- `plt.subplots(1, 2, ...)` — Creates **1 row, 2 columns** of subplots.
- `fig` — Figure object (whole canvas).
- `ax` — Array of two Axes objects: `ax[0]` left, `ax[1]` right.

```
sns.histplot(eda_df['word_len'], bins=50, kde=True, ax=ax[0])
ax[0].set_title('Word Count Distribution')
```

- `histplot` — Histogram of numeric column `word_len`.
- `bins=50` — Number of histogram buckets.
- `kde=True` — Overlays a smooth density curve.
- `ax=ax[0]` — Draw on the **left** subplot.

```
sns.boxplot(data=eda_df, x='label', y='word_len', ax=ax[1])
ax[1].set_title('Word Count by Class')
ax[1].tick_params(axis='x', rotation=20)
```

- `boxplot` — Shows median/quartiles/outliers of `word_len` **per** label.
- `x='label', y='word_len'` — Categories on x, numeric word length on y.
- `tick_params` — Rotates x labels on the **right** subplot only.

Word cloud

```
all_text = ' '.join(eda_df['clean_eda']).lower()
```

- `eda_df['clean_eda']` — All cleaned headlines as a Series.
- `' '.join(...)` — Concatenates every headline into **one** big string with spaces between them. **Why:** WordCloud expects one corpus string.
- `.lower()` — Lowercases everything so “Stock” and “stock” count as one. **Why:** Word-cloud frequency is usually case-insensitive.

```
wc = WordCloud(width=900, height=400, background_color='white').generate(a
```

- `WordCloud(...)` — Constructor: sets image size and background.
- `.generate(all_text)` — Builds frequency-based visualization from `all_text`.
- `wc` — Resulting WordCloud object (image data).

```
plt.figure(figsize=(14, 5))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('WordCloud of Training Headlines')
plt.show()
```

- `imshow` — Renders image array; `interpolation='bilinear'` smooths pixels.
 - `axis('off')` — Hides axes for a clean graphic.
-

Top 30 tokens

```
top_tokens = Counter(all_text.split()).most_common(30)
```

- `all_text.split()` — Splits giant string on whitespace → **list of word tokens** (no punctuation removed here—EDA uses lowercase join only).
- `Counter(...)` — Counts how often each token appears.
- `.most_common(30)` — Returns the 30 highest-frequency (token, count) pairs.

```
pd.DataFrame(top_tokens, columns=['token', 'count'])
```

- `pd.DataFrame` — Builds a table from the list of pairs.
 - `columns=['token', 'count']` — Names the two columns. **Why:** Readable table in notebook output (last expression often auto-displayed).
-

Mechanism of the whole cell

1. Define `strip_html_noise` for tag/entity/whitespace cleanup.
 2. Copy `train_df` to `eda_df` and add `clean_eda`, `char_len`, `word_len`.
 3. Plot class counts and word-length distributions; build a word cloud; list top tokens.
-

Role and importance in the notebook

EDA informs preprocessing choices and sequence length; it does not train models but prevents blind hyperparameter decisions later.